

Sreeraaghav Raja
3/8/26
40990068

EE Design 1 Technical Report

Microcontroller SPI & DAC

Introduction

This lab was focused on the Digital-to-Analog Converter (DAC) as opposed to the Analog-to-Digital Converter (ADC). The communication protocol used to communicate between the devices in this lab was the Serial-Peripheral Interface (SPI). The most important element about this module was understanding the use-cases behind the Digital-to-Analog Converter (DAC) and how to connect it to a microcontroller.

Methods

Background Information:

This lab was simple to do because of the wealth of information surrounding DACs and SPI. A DAC converts digital signals to analog signals through a variety of means, but the most basic one is an R-2R ladder which was covered in two labs in Circuits I. The R-2R ladder simply takes multiple digital signals and uses a resistor ladder with alternating resistors of value R and $2R$ to reduce the input signals and combine them into a specific output that will go from 0 to a pre-determined maximum voltage.

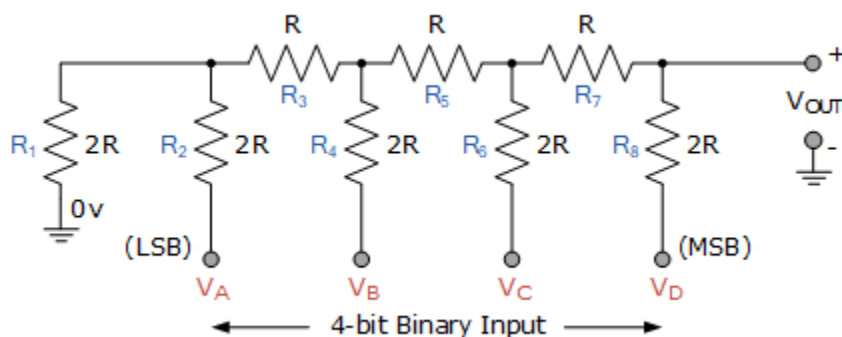


Figure 1: R-2R DAC Converter

The **4-bit R-2R ladder** in Figure 1 converts a 4-bit input signal into 16 possible output signals [1]. The mechanism behind this is that the inputs signals are either 0 V or X V where X is a

maximum voltage value. The input signals are digital so no matter what the maximum voltage is, they will always be treated as either 0 or 1 in binary. So, the DAC converts these input signals

into a standard output signal that will fit within the maximum voltage ($X V$). It is important to note that the process of taking a digital signal and converting it to an analog signal is often called **reconstruction** in signal processing [2]. Naturally, DACs are applied to audio processing because of how ubiquitous it is in every aspect of society. So, this lab is a natural progression into the audio processing rabbit hole.

SPI is another important mechanism that helped make this lab possible. SPI is a 4-line serial communication protocol that enables a microprocessor to communicate with an external device. SPI is full duplex so the master and slave devices can communicate with each other at the same time. This communication protocol does have a few interesting quirks though. SPI has individual lines for the Master to write to the Slave (MOSI) and the Master to read from the slave device (MISO), it has a chip select (CS) line that must be held when a microcontroller interacts with the target device, and it has 4 modes that must be chosen based on the mode of the target device. The 4 wires also allow SPI to be extremely fast compared to the other serial protocols at the disadvantage of being limited to the number of pins on the device and power requirements of having 4 lines [3].

The most important part of SPI for this lab was the 4 SPI modes which are shown in Figure 2. These 4 modes are determined by what the default value of the clock is, when the data is sampled, and when the data is shifted out. This table provides a comprehensive summary of each of these modes. This chart was particularly useful when determining the mode that the DAC uses to communicate.

Table 1. SPI Modes with CPOL and CPHA

SPI Mode	CPOL	CPHA	Clock Polarity in Idle State	Clock Phase Used to Sample and/or Shift the Data
0	0	0	Logic low	Data sampled on rising edge and shifted out on the falling edge
1	0	1	Logic low	Data sampled on the falling edge and shifted out on the rising edge
2	1	0	Logic high	Data sampled on the falling edge and shifted out on the rising edge
3	1	1	Logic high	Data sampled on the rising edge and shifted out on the falling edge

Figure 2: Table of SPI Modes

Figure 3 shows the timing diagram for the LTC1661, the DAC used in this module. It was simple to determine the SPI mode that this device uses after analyzing this diagram. Disregard the CS line for the sake of this explanation since that is trivial. The SCK/clock line is held low by default, so that eliminates SPI Modes 2 and 3 since they are high by default. The DIN line is the input data, and each packet of data is held until a rising edge and then shifted out by the falling edge of each SCK pulse. Therefore, the DAC utilizes SPI Mode 0 and that was used to initialize the SPI module in the code [4].

TIMING DIAGRAM

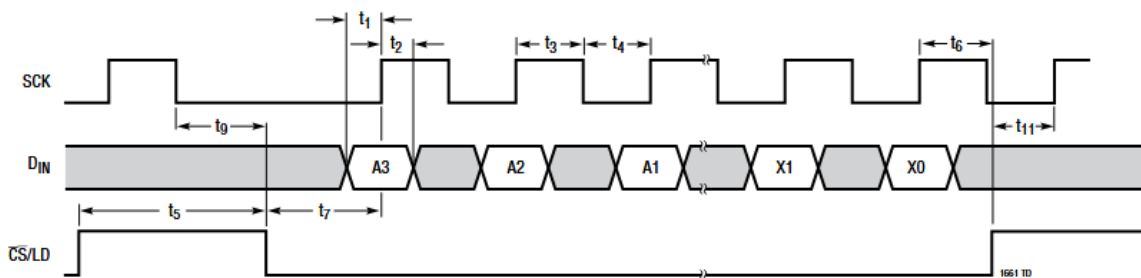


Figure 3: LTC1661 Timing Diagram

Design Steps:

The design steps followed the same dogma as every other lab: **design the circuit before writing any code**. The circuit was easy to design because of how every lab was built off the previous ones. Therefore, I will only explain aspects that were different for this lab compared to the previous labs regarding the circuit and code design. Figures 4 and 5 are the circuit and hardware block for this lab. The difficult part of this circuit was realizing that the MISO line can remain disconnected because the Pico never needs to read from the DAC since the device should convert inputs signals into specific outputs. Additionally, a key realization that I made was that the VCC pin on the DAC should be connected to 5V since the output signals should range from 0V to 5V or else the signal will clip.

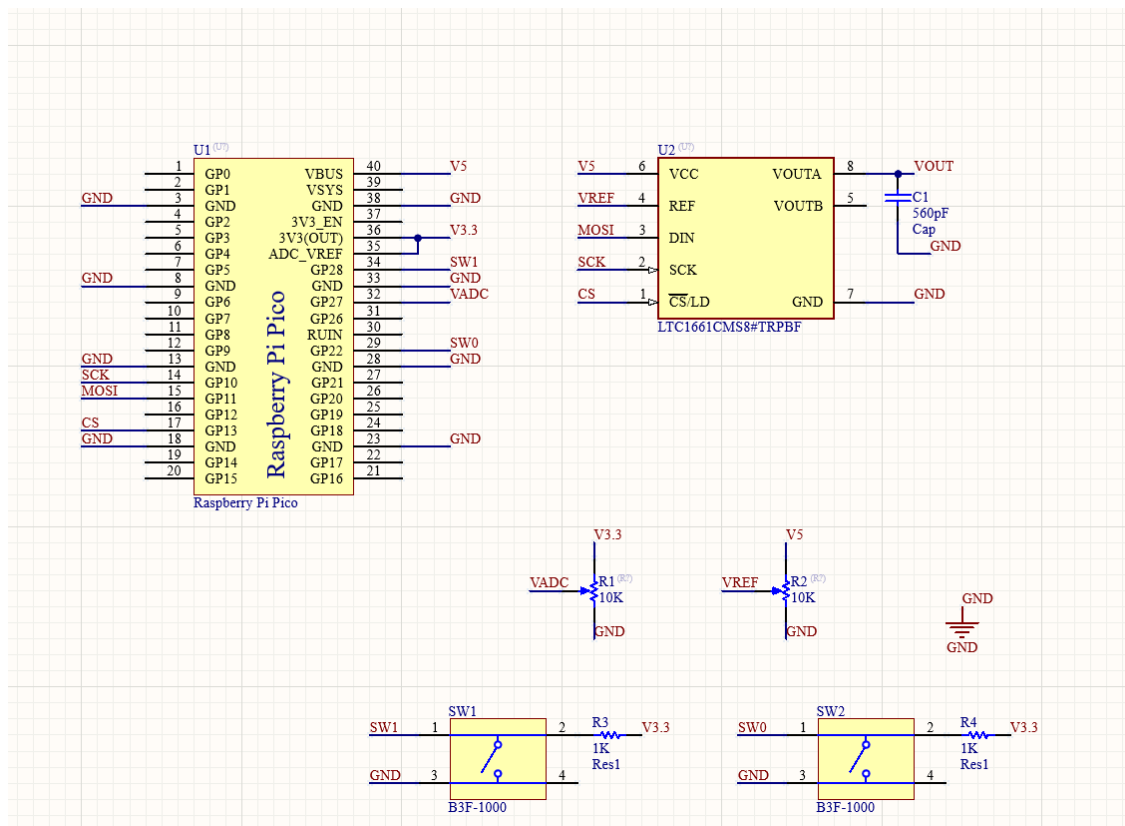


Figure 4: Altium Circuit Design

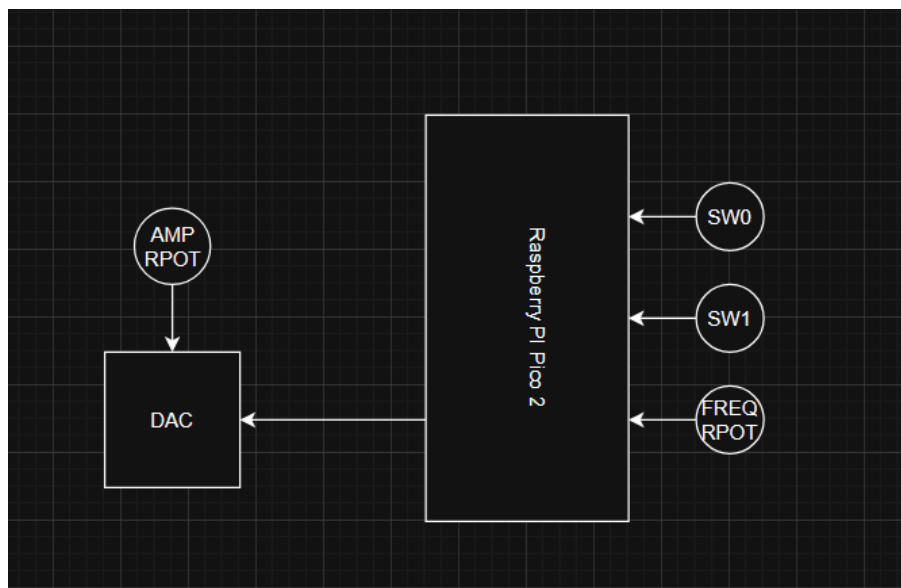


Figure 5: Hardware Block

Since the lab requires DAC to output signals at multiple frequencies in real time, the key design method is to reduce the latency of the code. Therefore, I put a minimal amount of code in the while loop and had a reasonable sleep timer to align my design with that constraint. The code is separated into 4 different pseudo-simultaneous parts: the main loop, DAC read timer loop, the frequency potentiometer read timer loop, and the button interrupt and debounce timer.

Figure 6 shows a breakdown of the main loop of the code. The blocks start with initializing all the necessary modules like the SPI module, the 4 timers (DAC read timer, the Frequency Potentiometer timer, and the 2 one-shot debounce timers). The initialization block also initializes any global flags, the byte array, ADC frequency conversion variables, and the DAC frequency. In the while loop, the program checks to see if a switch's value has changed, if it has, then the program checks which switch changed and the corresponding wave is output, and the switch values are print before restarting the loop.

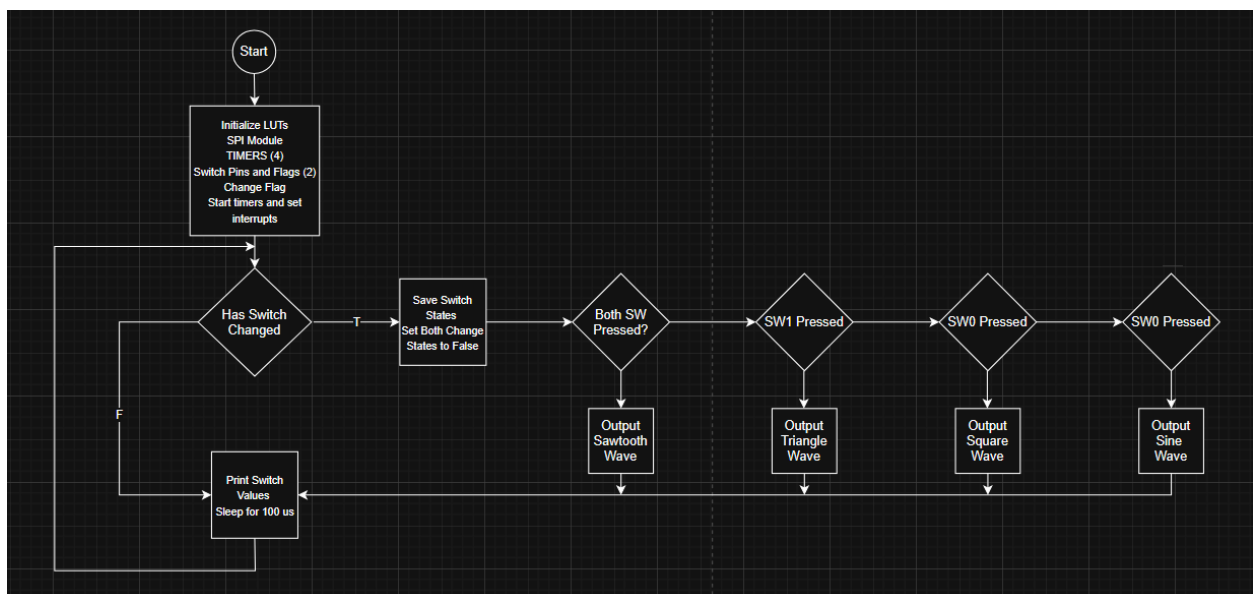


Figure 6: Code Block 1

Figure 7 shows the three other simultaneous programs. The **DAC timer** essentially packs the array data from the LUT and a control word that activates the DAC. Then, the program sends that data to the DAC and increments the LUT index by 1. The ADC Frequency Timer is to read and convert the value from the frequency potentiometer. This timer is much slower than the DAC timer since the Potentiometer is operated by a human. This program waits for the current ADC value that is read to be much greater than the previous ADC value. If this is the case, then we update the DAC timer with the new frequency value after converting that currently read frequency value. Finally, the switch interrupts block waits for a switch to activate

before disabling that interrupt, setting all necessary flags, and starting a one-shot debounce timer. After that time has passed, the interrupt is re-enabled.

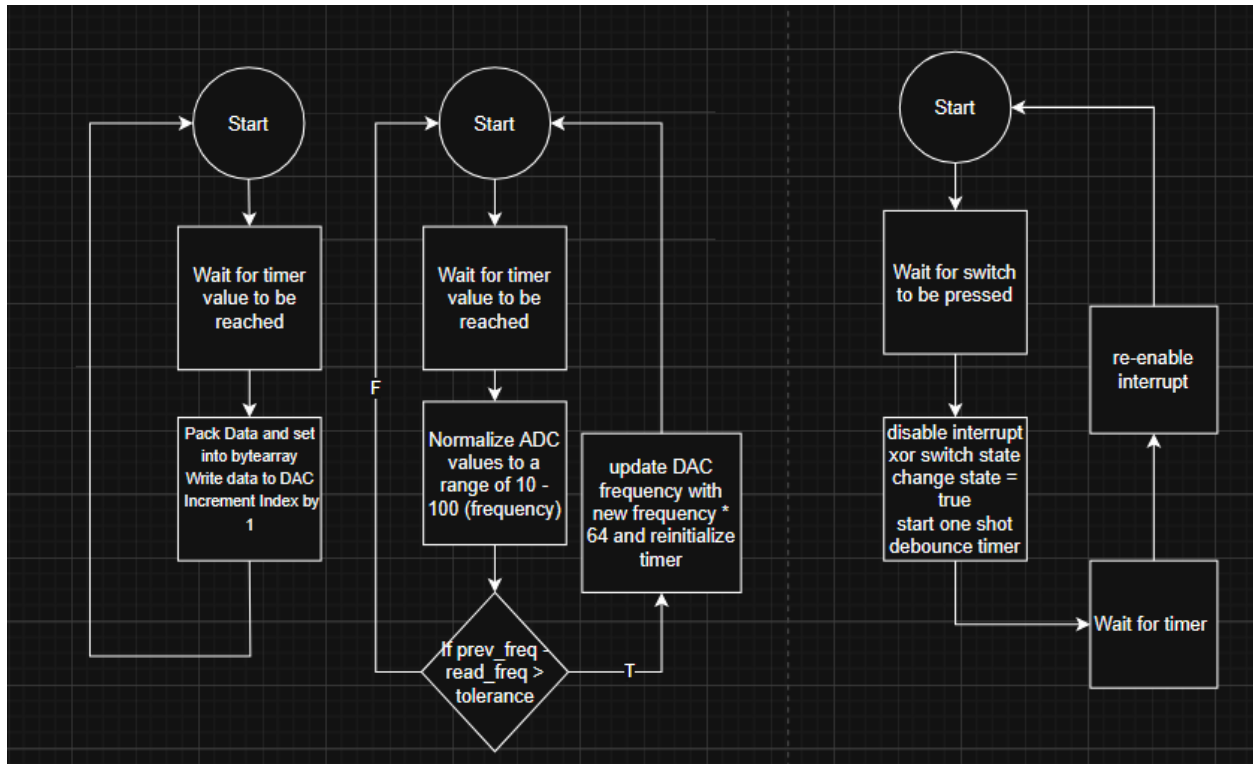


Figure 7: Code Block 2

Overall, the design order that I usually follow allowed me to enjoy this lab a lot more than the previous labs. My design process was thorough, and I felt as though each segment was built to the next one well. I was also able to find solutions to timing issues easily using the Digilent Analog Discovery (DAD) Board, so any issues I had with either SPI or the wiring were easy to solve.

Results

The results matched the design specifications on the second attempt. The first attempt had a minor issue with signal clipping that because of connecting the VCC pin of the LTC1661 DAC to the 3.3V line instead of the 5V line. Nevertheless, the four different waves were the sine, square, sawtooth, and triangle waves that could range from 0V to 5V and oscillate at a frequency between 10Hz and 100Hz.

Figure 8 is an example of the amplitude manipulation for this module. The potentiometer is connected to the Ref pin on the DAC, so turning it from 0 to 5 volts is effective for the amplitude manipulation. Figure 8 shows a 0V sine wave, so the oscilloscope shows nothing.

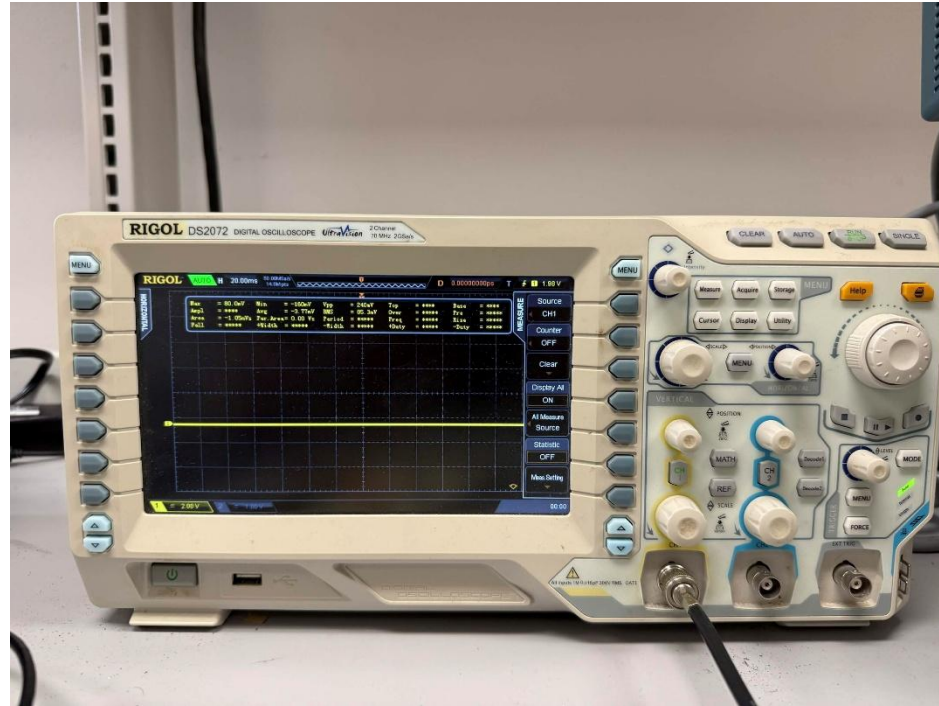


Figure 8: 0V Amplitude Sine Wave Output

Figures 9 through 12 show the different waves required as outputs for this module. Each wave is set at its maximum frequency of 100Hz and maximum voltage of 5V to ensure that the waves can meet the required specifications. Interestingly, the high value of the 5V signal will decrease in the range of mV which could be related to the capacitor at the output terminal's charging and discharging cycles.

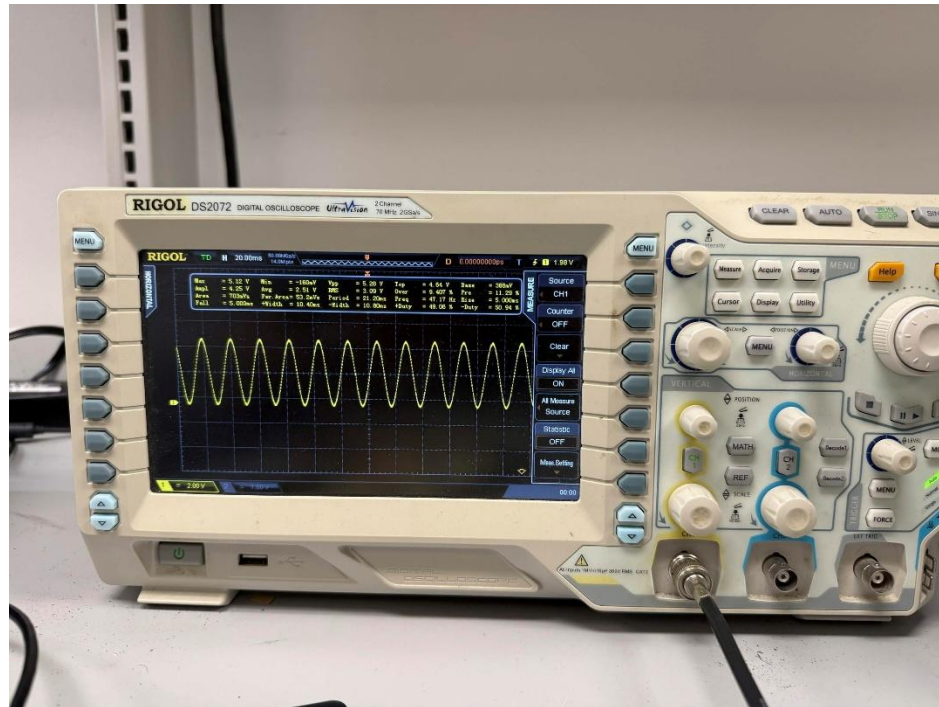


Figure 9: Sine Wave Output

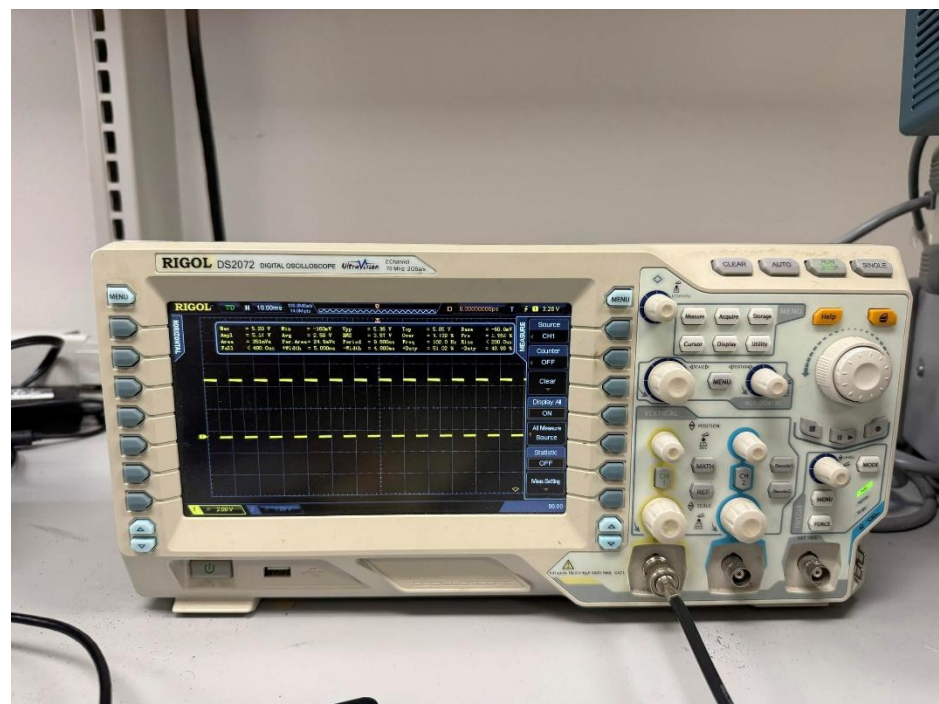


Figure 10: Square Wave Output

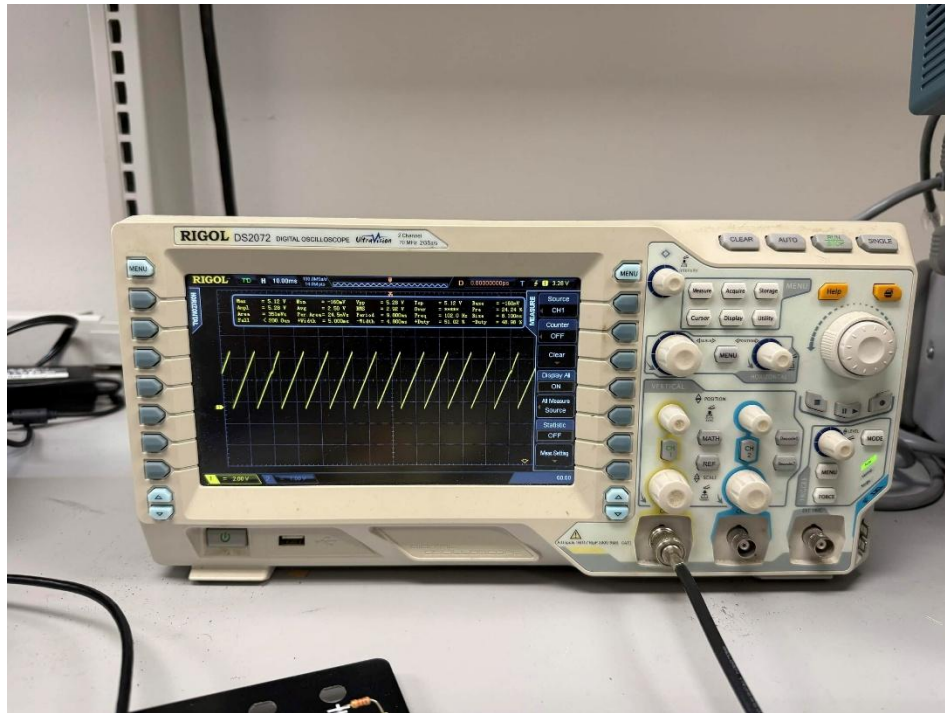


Figure 11: Sawtooth Wave Output

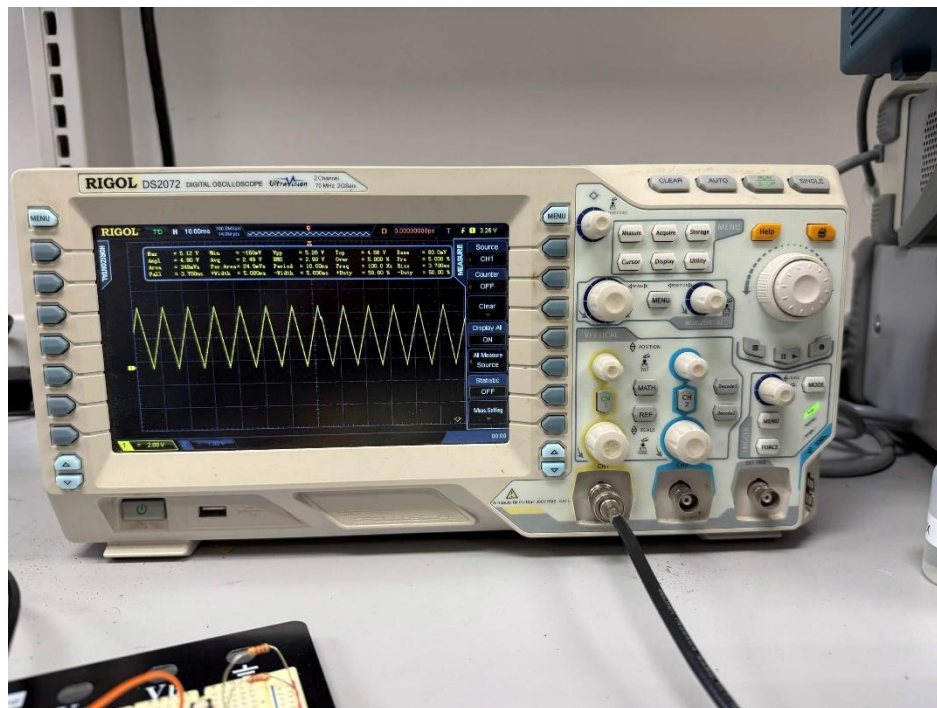


Figure 12: Triangle Wave Output

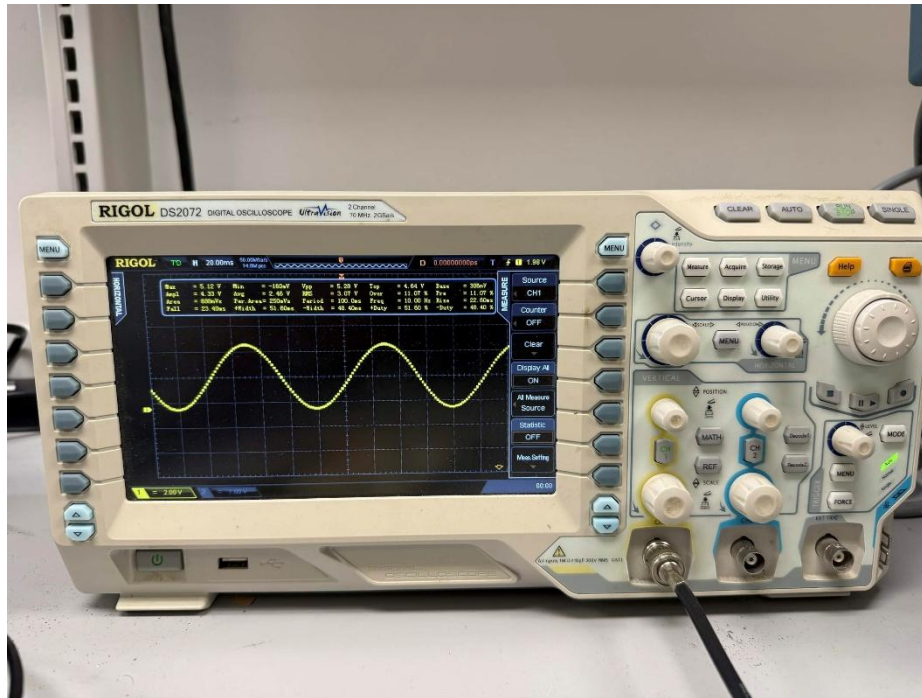


Figure 13: 10Hz Sine Wave Output

Figure 13 shows a 10Hz Sine Wave Outputs which is an example of the frequency oscillation of the signal. The mechanism I used to change frequencies was to read the output of a potentiometer that was connected to 3.3V and ground. I used an ADC to convert that data into 16-bits in MicroPython and normalized those values from 0 to 1. Then, I multiplied that value by 90 and added 10 to get the frequency values in the range of 10Hz to 100Hz. The primary mechanism of changing the specific DAC read frequency was already discussed in the explanation of the code block diagram in the Methods section.

Conclusion

This lab helped familiarize me with an external DAC module, something that I have never used before. I already knew how to utilize SPI with other microcontrollers like the Tiva board, but this lab was useful practice and helped me dive into the theory even more. It felt like my eyes were tied to anchors when I was reading the DAC documentation, but the end product was satisfying.

Appendix

```
from machine import SPI, ADC, Pin, Timer
from time import sleep
import utime

# ----- LUTs ----- #
sine_lookup_table = (
    512, 562, 611, 660, 707, 753, 796, 836,
    873, 907, 937, 963, 984, 1001, 1013, 1021,
    1023, 1021, 1013, 1001, 984, 963, 937, 907,
    873, 836, 796, 753, 707, 660, 611, 562,
    512, 461, 412, 363, 316, 270, 227, 187,
    150, 116, 86, 60, 39, 22, 10, 2,
    0, 2, 10, 22, 39, 60, 86, 116,
    150, 187, 227, 270, 316, 363, 412, 461
)

square_lookup_table = 32 * (0,) + 32 * (1023,)

triangle_lookup_table = (
    32, 64, 96, 128, 160, 192, 224, 256,
    288, 320, 352, 384, 416, 448, 480, 512,
    543, 575, 607, 639, 671, 703, 735, 767,
    799, 831, 863, 895, 927, 959, 991, 1023,
    991, 959, 927, 895, 863, 831, 799, 767,
    735, 703, 671, 639, 607, 575, 543, 512,
    480, 448, 416, 384, 352, 320, 288, 256,
    224, 192, 160, 128, 96, 64, 32, 0,
)

sawtooth_lookup_table = (
    0, 16, 32, 48, 64, 81, 97, 113,
    129, 146, 162, 178, 194, 211, 227, 243,
    259, 276, 292, 308, 324, 341, 357, 373,
    389, 406, 422, 438, 454, 471, 487, 503,
    519, 536, 552, 568, 584, 601, 617, 633,
    649, 666, 682, 698, 714, 731, 747, 763,
    779, 796, 812, 828, 844, 861, 877, 893,
    909, 926, 942, 958, 974, 991, 1007, 1023
)

# ----- LUTs ----- #
```

```

LUT_LENGTH = 64
CONTROL_WORD = 0xF # this loads new data into input reg A and sends that to the

freq_pot = ADC(27)          # connect to ADC1
conversion_factor = 3.3 / (65535)
conversion_factor_norm = conversion_factor / 3.3

count : int = 0
index : int = 0

# ----- INITIALIZATION ----- #
# initialize SPI module
spi = SPI(1, baudrate = 400000, polarity = 0, phase = 0, bits = 8, firstbit =
SPI.MSB)
# set chip select separately to toggle and set it as an output pin with default
value high (Active-Low CS)
cs = Pin(13, Pin.OUT, value = 1)

# initialize timers
tim = Timer()
# timer 2 for the freq pot
tim2 = Timer()

# initialize 16-bit array to store data that we're sending - 2 bytes
send = bytearray(2)
# choose wave_type
wave = sine_lookup_table

# initialize the dac update timer
samp_freq = 100
dac_update_freq = samp_freq * 64
tolerance = 1

# initialize Pins for switches
sw1 = Pin(28, mode=Pin.IN, pull=Pin.PULL_UP)
sw0 = Pin(22, mode = Pin.IN, pull=Pin.PULL_UP)
# switch flag
sw_state = [0,0]
change_state = [False,False]

# initialize 2 one-shot timer objects
tim3 = Timer()

```

```

tim4 = Timer()

# ----- INITIALIZATION ----- #

def pack_data(data):
    # pack data and bottom two bits are don't cares, so don't matter
    return ((CONTROL_WORD << 12) | ((data & 0x3FF) << 2))

def update_dac(timer):
    global index

    packet = pack_data(wave[index])

    send[0] = (packet >> 8) & 0xFF
    send[1] = packet & 0xFF

    # choose DAC as device
    cs.low()
    spi.write(send)
    cs.high()

    index = (index + 1) % 64

def update_freq(timer):
    global dac_update_freq
    global samp_freq

    freq_reading = round(freq_pot.read_u16() * conversion_factor_norm * 90 + 10)
    if abs(samp_freq - freq_reading) > tolerance:
        samp_freq = round(freq_reading)
        dac_update_freq = samp_freq * 64
        tim.init(mode = Timer.PERIODIC, freq = dac_update_freq, callback =
update_dac)

def sw0_irq_handler(Pin):
    sw0.irq(handler=None)
    sw_state[0] = sw_state[0] ^ 1
    change_state[0] = True
    # start a one shot timer
    tim3.init(mode=Timer.ONE_SHOT, period=300, callback=sw0_callback)

def sw1_irq_handler(Pin):
    sw1.irq(handler=None)
    sw_state[1] = sw_state[1] ^ 1

```

```

change_state[1] = True
# start a one shot timer
tim4.init(mode=Timer.ONE_SHOT, period=300, callback=sw1_callback)

def sw0_callback(timer):
    # re-enable the interrupt
    sw0.irq(handler=sw0_irq_handler, trigger=Pin.IRQ_FALLING)

def sw1_callback(timer):
    # re-enable the interrupt
    sw1.irq(handler=sw1_irq_handler, trigger=Pin.IRQ_FALLING)

def main():
    # initialize callback timers
    tim.init(mode = Timer.PERIODIC, freq = dac_update_freq, callback =
update_dac)
    tim2.init(mode = Timer.PERIODIC, freq = 20, callback = update_freq)

    # enable the switch interrupts
    sw0.irq(handler=sw0_irq_handler, trigger=Pin.IRQ_FALLING)
    sw1.irq(handler=sw1_irq_handler, trigger=Pin.IRQ_FALLING)

    while True:
        global index, wave
        if change_state[0] or change_state[1]:
            sw0_active = sw_state[0]
            sw1_active = sw_state[1]
            change_state[0] = False
            change_state[1] = False

            if sw0_active and sw1_active:
                wave = sawtooth_lookup_table
            elif sw1_active:
                wave = triangle_lookup_table
            elif sw0_active:
                wave = square_lookup_table
            else:
                wave = sine_lookup_table

            index = 0
            print(f"SW0: {sw_state[0]}, SW1: {sw_state[1]}")

            utime.sleep_ms(100)

main()

```

References

[1] "What is a DAC (Digital-to-Analog Converter) and How Does it Work?," *Audio Advice*, 2024. <https://www.audioadvice.com/blogs/expert-advice/what-is-a-dac-how-does-it-work> (accessed Mar. 09, 2026).

[2] ElectronicsTutorials, "R-2R Digital-to-Analogue Converter (R-2R DAC)," *Basic Electronics Tutorials*, Oct. 13, 2020. <https://www.electronics-tutorials.ws/combination/r-2r-dac.html> (accessed Mar. 09, 2026).

[3] P. Dhaker, "Introduction to SPI Interface | Analog Devices," *Analog.com*, Sep. 2018. <https://www.analog.com/en/resources/analog-dialogue/articles/introduction-to-spi-interface.html> (accessed Mar. 09, 2026).

[4] "LTC1661 Datasheet and Product Info | Analog Devices," *Analog.com*, 2016. <https://www.analog.com/en/products/ltc1661.html> (accessed Mar. 09, 2026).